Extending LMCache's Prefix-Cache Eviction Policy: Cost-Awareness and Windowed Admission Control

Design, Evaluation, and Ablation Report

Project: LMCache -- prefix-cache eviction policy extension

Repository: LMCache (branch: cost-aware-policy)

Author: Aya Neeman

Date: July 30, 2026

Abstract. LMCache's storage backends select which cached KV chunks to evict using simple recency/frequency baselines (LRU, LFU, FIFO, MRU), which are blind to the heterogeneous recompute cost of different chunks and to the difference between a low-value and a high-value newcomer. This report evaluates three extensions built and shipped for lmcache/v1/storage_backend/cache_policy/: (A) a cost- and frequency-aware scoring policy, (B) a TinyLFU-style admission-control wrapper that can reject low-value newcomers outright, and (C) a windowed variant of (B) designed to remove a correctness limitation discovered while evaluating it. Across a synthetic benchmark suite and a statistically validated real-data (ShareGPT conversation) evaluation, admission control is the strongest general-purpose improvement, cost-awareness helps synthetic but not real traffic, and the windowed variant trades some peak hit rate for eliminating a catastrophic freeze failure mode under one-shot-dominated traffic. No single design dominates on every axis; both admission-control variants ship as independently selectable policies.

1. Introduction

LMCache accelerates LLM inference by caching key-value (KV) tensors for previously seen token prefixes, so a later request that shares a prefix (a repeated system prompt, an earlier turn of the same conversation, a common few-shot template) can reuse cached KV instead of recomputing it. Under memory pressure, a cache-eviction policy decides which cached chunks to discard to make room. LMCache ships this decision behind a small, swappable BaseCachePolicy interface (lmcache/v1/storage_backend/cache_policy/), with four baseline implementations available out of the box: LRU, LFU, FIFO, and MRU.

These four baselines share a blind spot: they rank cached chunks purely by recency or by raw access count, and they have no way to refuse a newcomer -- eviction always makes room for whatever arrives next, regardless of whether the newcomer is likely to be reused. Two properties of real inference workloads make this a real cost, not a theoretical one. First, cached chunks are not equally expensive to regenerate on a miss: a long, expensive-to-recompute prefix and a short, cheap one occupy the same eviction priority under LRU purely because of when they were last touched. Second, in genuinely popularity-skewed or bursty traffic, a policy that always admits every newcomer will happily evict a frequently-reused chunk to make room for a chunk that will likely never be touched again.

This motivates two independent ideas, both well established in the caching literature outside of LLM serving: cost-aware eviction (scoring candidates by a combination of expected reuse and recompute cost, in the tradition of GreedyDual-Size-style web-cache policies) and frequency-gated admission control (rejecting a newcomer outright instead of always evicting an incumbent, in the tradition of the TinyLFU / Window-TinyLFU design used by Caffeine, a widely deployed JVM caching library). This project ports both ideas into LMCache's cache_policy abstraction, builds a CPU-only benchmark harness that replays the real policy call sequence a storage backend makes, and evaluates the resulting policies on both synthetic workloads and a real multi-turn conversation corpus (ShareGPT). The central question this report answers is not just "does the new policy beat the baseline on one benchmark," but whether the improvement is general -- does it hold across cache sizes, does it hold on real traffic and not just synthetic traffic, and does it introduce any new failure modes the baseline didn't have.

2. Extension Design

2.1 Baseline

BaseCachePolicy defines the contract every policy implements: init_mutable_mapping (construct the cache's backing dict), update_on_hit / update_on_put (record an access or insertion), get_evict_candidates (rank and return keys to remove under pressure), and update_on_force_evict (cleanup hook). LRU, LFU, FIFO, and MRU each implement this with O(1)-amortized bookkeeping (an OrderedDict for LRU/FIFO/MRU, a frequency-bucketed SortedDict for LFU) and serve as both the production defaults and the baseline this report measures every extension against.

2.2 Direction A -- CostAwareEvictionPolicy

Scores each resident chunk by combining three signals: an EWMA-smoothed cost density (observed/estimated recompute cost divided by the chunk's memory footprint, so a large expensive chunk and a small cheap chunk with the same cost-per-byte are treated equally), an exponential recency

decay (half-life configurable), and a log-dampened access-frequency term. The frequency term was added specifically to fix an initial cost-only version that lost to plain LRU/LFU on real ShareGPT data: with no frequency signal, cost-density alone could keep an expensive chunk resident indefinitely even after it stopped being reused. get_evict_candidates ranks by this score ascending (lowest score evicted first), with untrusted (no cost metadata yet) candidates evicted before any fully scored candidate.

2.3 Direction B -- AdmissionControlledPolicy

Wraps any inner policy (LRU, LFU, FIFO, MRU, or CostAware) and adds one new hook, should_admit(key, cache_dict), called only when the cache is already full. It maintains its own decaying frequency sketch (a plain dict with periodic halving, not a real Count-Min Sketch -- collisions are not the object of study here) and admits a newcomer only if its estimated frequency exceeds the coldest currently-resident key's estimate; otherwise the newcomer is rejected and the incumbent keeps its slot. The inner policy is untouched and continues to own eviction ranking for whatever admission control lets through -- get_cache_policy("ADMISSION_<INNER>") composes with any registered policy name.

Building this as a real, tested class (rather than trusting the prototype that motivated it) surfaced two genuine bugs, both caught by re-running the evaluation rather than by inspection: (1) the first version didn't record a frequency observation for a rejected key, so a key that lost its first admission bid could never win a later one -- a permanent lockout, caught because the shipped class scored worse than plain LRU, the opposite of the validated prototype's result; (2) the first version speculatively called the inner policy's get_evict_candidates just to compare frequencies, which corrupted LFUCachePolicy's internal bookkeeping when the speculative peek was discarded (LFU mutates its state as a side effect of that call, not of a separate evict step), caught by a KeyError crash the first time LFU was wrapped. Both are documented in full in admission-control-policy.md and are fixed in the shipped class.

2.4 Direction C -- WindowedAdmissionControlledPolicy

Evaluating Direction B to the same depth as Direction A (Section 4) surfaced a real design limitation: should_admit's strict greater-than comparison always favors the incumbent on a tie, which is exactly right under heavy eviction pressure but has two failure modes elsewhere -- a real hit-rate regression under generously-sized, low-pressure caches, and a permanent, silent freeze under purely one-shot traffic (every newcomer's frequency estimate of 1 never strictly exceeds an incumbent's, so nothing is ever admitted again after the first fill). Rather than modify the shipped class, this was fixed as a second, independently selectable policy so both designs remain directly comparable. WindowedAdmissionControlledPolicy always admits new keys into a small, bounded, always-admits window; only when the window itself overflows is its oldest member evaluated -- promoted into the frequency-gated main region if it clears a promotion threshold, or queued for a real eviction otherwise. Because the window never rejects, the freeze failure mode becomes structurally unreachable.

The first implementation of this design had its own bug, caught the same way Bug 2 was caught -- an implausible exact match between two supposedly different policies' eviction counts. Window capacity was computed lazily from cache_dict's size and pruned only during eviction, which let the window grow to the entire cache during the fill phase (nothing prunes it before the first eviction) and then never shrink (every post-fill eviction cycle removed one window member and unconditionally added one new key back -- a mathematically invariant wash). The fix makes window_capacity an absolute integer enforced immediately at insertion time, with a pending-discard queue standing in for the eviction authority that insertion-time code does not have. Section 5.3 revisits this bug as a worked example of the report's evaluation methodology.

3. Experimental Setup

3.1 Tooling

lmcache/tools/cache_policy_bench/runner.py implements a CPU-only simulator (_PolicyCache) that drives a real policy object through the exact call sequence a storage backend makes on every request: should_admit (when full) before insertion, update_on_hit for prefix hits, update_on_put_with_metadata for insertions, and get_evict_candidates under capacity pressure. A CostModel maps hit-prefix length and recompute-token count to a modeled latency (no GPU or running model required), from which hit rate, eviction count, rejected-admission count, and latency percentiles are aggregated per run.

3.2 Workloads

- repetitive_short -- small vocabulary, high reuse density.
- novel_long -- long, unique documents touched exactly once; the purely one-shot stress case for admission control.
- mixed_zipfian -- Zipf-distributed popularity over a prefix pool; skew strength (zipf_s) is a swept parameter.
- multi_round_chat -- simulated multi-turn sessions with growing shared prefixes, approximating conversational reuse.
- Real ShareGPT corpus -- roughly 35,000 real multi-turn conversations (via the existing benchmarks/multi_round_qa/ download and tokenization pipeline), adapted into the same Request shape the synthetic generators produce. This is the only workload not authored for this project.

3.3 Metrics and statistical method

Primary metrics: token hit rate, eviction count, rejected-admission count, and modeled latency p50/p95/p99. For the real-data evaluation, every (policy, corpus-scale, cache-size) cell is run 6 times with a fresh bootstrap resample of the conversation corpus (dependency-free percentile-bootstrap implementation in benchmarks/cache_policy/stats.py), and results are reported as a mean with a 95% confidence interval rather than a single point estimate -- a difference that matters directly in Section 4.3.

3.4 Parameters swept

- Cache size: 50, 100, 200 MiB (synthetic and real-data sweeps).
- Corpus scale: 500, 2,000, 5,000 conversations (real-data only).
- Zipf skew: zipf_s in {0.6, 1.2, 2.0} (mild to extreme).
- halve_every (frequency-sketch decay window, both admission-control designs): 2,000 / 20,000 / 200,000.
- window_capacity (Direction C): 5 / 20 (default) / 80.
- promotion_threshold (Direction C): 1 / 2 (default) / 4.

4. Results

4.1 Synthetic cache-size sweep: vanilla vs. extended

Figure 1 sweeps hit rate across all three cache sizes for five representative policies (baseline LRU and LFU, plus the three extensions) on all four synthetic workloads. Figure 2 shows the corresponding

modeled p95 latency for the two workloads with the most eviction pressure. Table 1 gives the exact numbers at 100 MiB, the sweep's middle cache size, with hit-rate deltas and relative p95-latency change against the plain-LRU baseline.

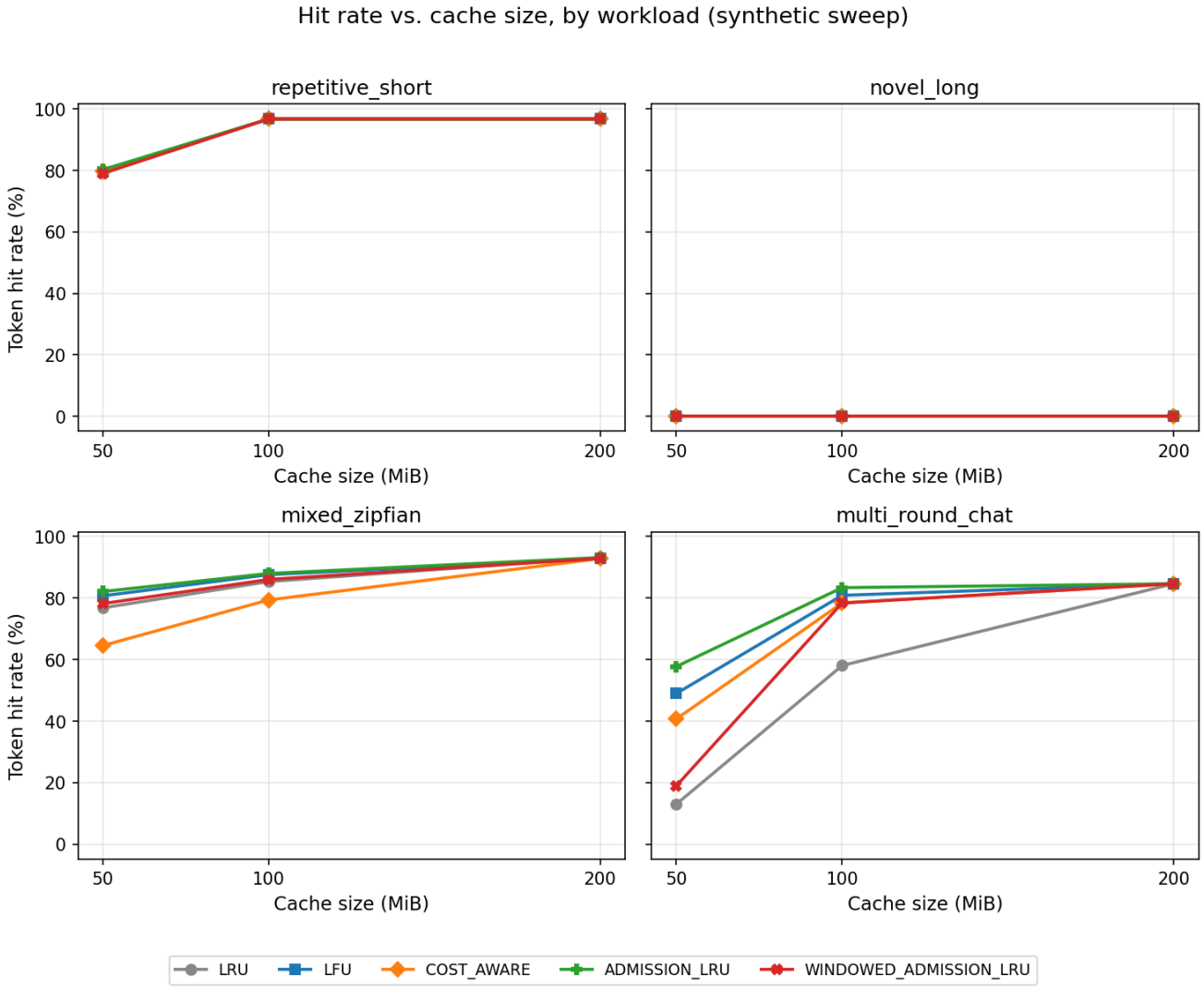

Figure 1. Token hit rate vs. cache size, by workload. All four workloads, 50/100/200 MiB. novel_long is 0% for every policy by construction (no chunk is ever touched twice).

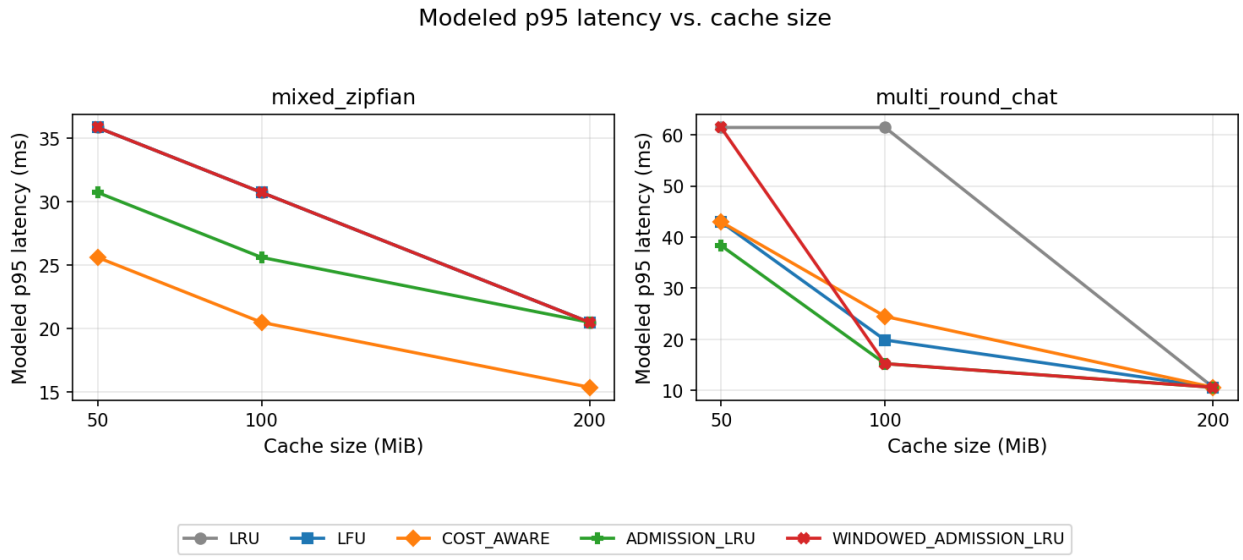

Figure 2. Modeled p95 latency vs. cache size, mixed_zipfian and multi_round_chat -- lower is better.

Table 1. Hit rate and modeled p95 latency at 100 MiB, relative to plain LRU.

Workload	Policy	Hit rate	vs. LRU	Evictions	p95	vs. LRU
mixed_zipfian	LRU (baseline)	85.3%	--	1,973	30.7ms	--
mixed_zipfian	LFU	87.6%	+2.2pp	1,611	30.7ms	+0.0%
mixed_zipfian	COST_AWARE	79.3%	-6.0pp	2,938	20.5ms	-33.3%
mixed_zipfian	ADMISSION_LRU	88.0%	+2.6pp	288	25.6ms	-16.7%
mixed_zipfian	WINDOWED_ADMISSION_LRU	86.0%	+0.7pp	1,862	30.7ms	+0.0%
multi_round_chat	LRU (baseline)	58.0%	--	910	61.4ms	--
multi_round_chat	LFU	80.8%	+22.8pp	198	19.9ms	-67.7%
multi_round_chat	COST_AWARE	78.4%	+20.4pp	236	24.5ms	-60.2%
multi_round_chat	ADMISSION_LRU	83.3%	+25.3pp	0	15.2ms	-75.2%
multi_round_chat	WINDOWED_ADMISSION_LRU	78.3%	+20.3pp	227	15.2ms	-75.2%

ADMISSION_LRU has the highest hit rate of every policy tested on both workloads shown, and cuts p95 latency by 16.7% and 75.2% respectively -- on multi_round_chat it drives eviction count to exactly zero, meaning the working set stabilizes entirely rather than continuing to churn. WINDOWED_ADMISSION_LRU recovers most, but not all, of this improvement -- a pattern this report returns to repeatedly. COST_AWARE actually loses to plain LRU on mixed_zipfian even though it wins decisively on multi_round_chat, consistent with a design tuned around cost heterogeneity, not raw popularity skew.

4.2 Why hit rate improves: evictions vs. rejections

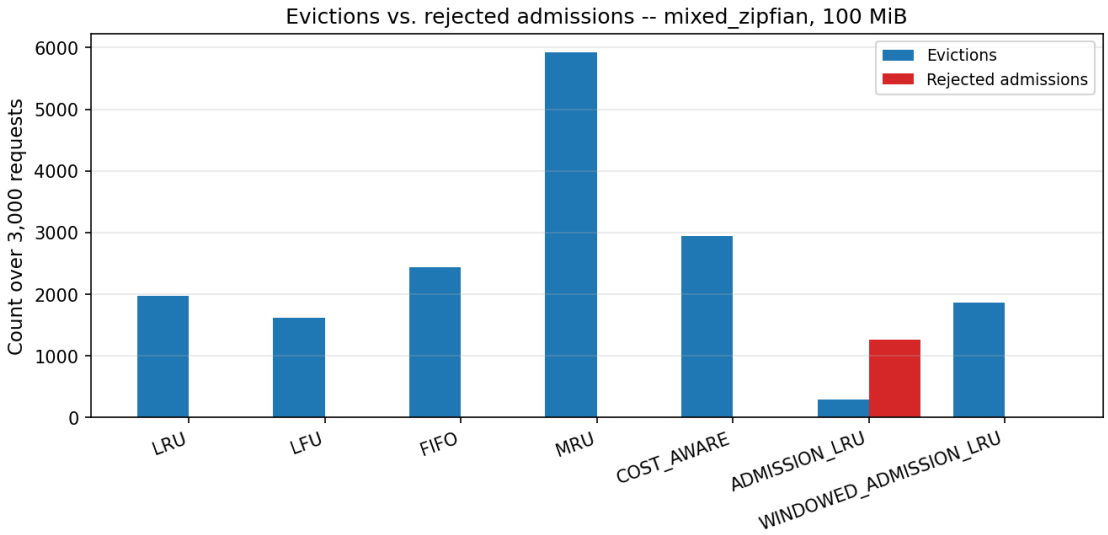

Figure 3. Evictions vs. rejected admissions, mixed_zipfian, 100 MiB, 3,000 requests. Admission-controlled policies convert most churn into rejections instead of evictions.

Figure 3 makes the mechanism concrete: ADMISSION_LRU's 288 evictions at 100 MiB replace what would otherwise be roughly 1,973 evictions (LRU's count) with 1,260 outright rejections -- the newcomers that would have displaced a warmer incumbent are simply never let in. WINDOWED_ADMISSION_LRU shows the structural difference underlying Direction C: its rejected-admission count is always zero by design (Section 2.4) -- churn is entirely expressed as evictions from the window, at a rate closer to plain LRU's.

4.3 Real-data validation (ShareGPT)

Figure 4 shows hit rate with 95% bootstrap confidence intervals at 200 MiB across three corpus scales, split into an LRU family (LRU, ADMISSION_LRU, WINDOWED_ADMISSION_LRU) and a COST_AWARE family. Table 2 gives the full LRU-family grid across all three cache sizes and three scales tested.

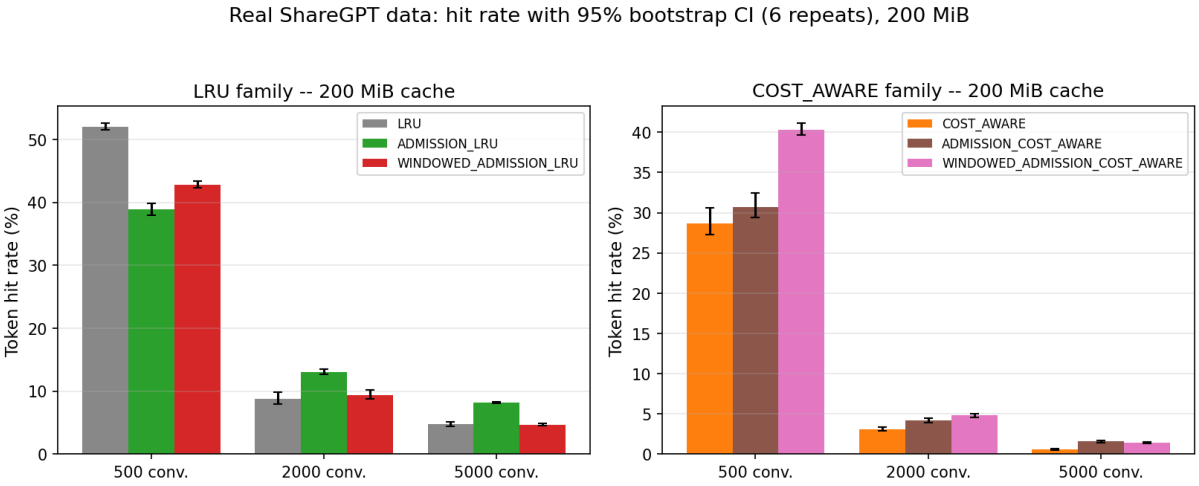

Figure 4. Real ShareGPT hit rate with 95% bootstrap CI (6 repeats), 200 MiB, by corpus scale. Error bars are the bootstrap 95% CI, not standard error.

Table 2. Real ShareGPT mean hit rate (6-repeat bootstrap), LRU family, all cells.

Scale	Cache	LRU	ADMISSION_LRU	WINDOWED_ADMISSION_LRU
500	50 MiB	10.0%	13.8%	10.9%
500	100 MiB	18.0%	23.6%	24.1%
500	200 MiB	52.1%	38.9%	42.8%
2,000	50 MiB	3.2%	4.9%	3.3%
2,000	100 MiB	5.2%	7.9%	5.5%
2,000	200 MiB	8.8%	13.1%	9.4%
5,000	50 MiB	1.6%	3.2%	1.7%
5,000	100 MiB	2.8%	5.1%	2.8%
5,000	200 MiB	4.8%	8.2%	4.7%

ADMISSION_LRU wins 8 of 9 cells, several by a wide margin with non-overlapping confidence intervals -- a genuine, statistically supported effect, not sampling noise. The one loss (highlighted row, 500 conversations / 200 MiB, the most generously sized cache relative to its working set) is the real-data confirmation of the regression predicted from the strict tie-breaking rule in Section 2.4: ADMISSION_LRU drops 13.2 percentage points below plain LRU. WINDOWED_ADMISSION_LRU recovers about a third of that gap (42.8% vs. 38.9%) at this cell, and is competitive with or slightly better than the strict design at 100 MiB and below -- but at the larger, more one-shot-dominated scales (2,000 and 5,000 conversations), it converges to plain LRU rather than to ADMISSION_LRU's wins, because almost nothing reaches the promotion threshold before its first eviction opportunity under traffic this close to purely one-shot.

4.4 Robustness across Zipf skew

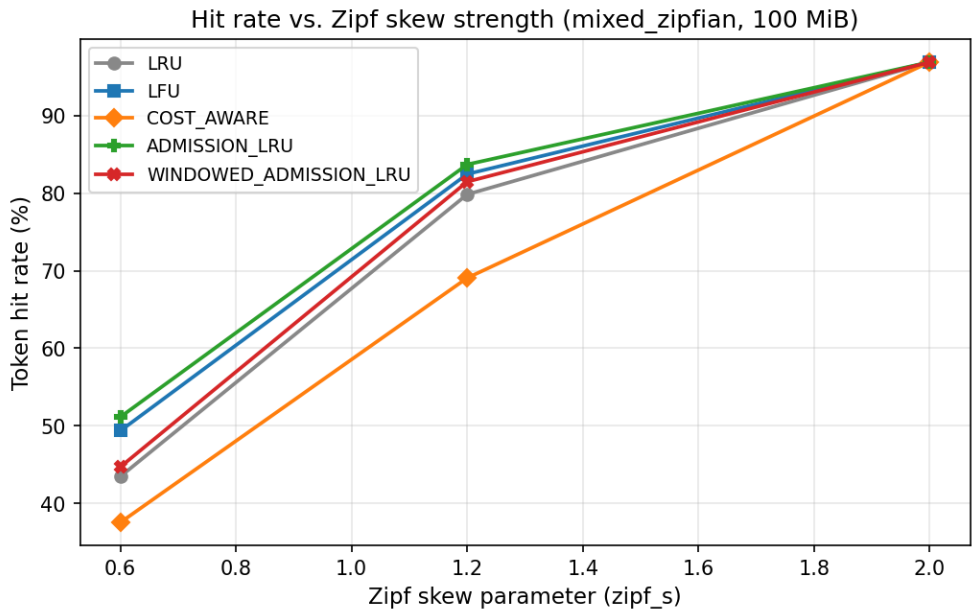

Figure 5. Hit rate vs. Zipf skew strength, mixed_zipfian, 100 MiB. All policies converge once the working set fits entirely in cache (zipf_s = 2.0, zero evictions for every policy).

ADMISSION_LRU has the highest hit rate at both mild (zipf_s=0.6) and default (zipf_s=1.2) skew, confirming Section 4.1's result holds across skew strength rather than being an artifact of one parameter value. WINDOWED_ADMISSION_LRU again lands between the baseline and the strict design at every point tested.

4.5 Latency variability across repeats

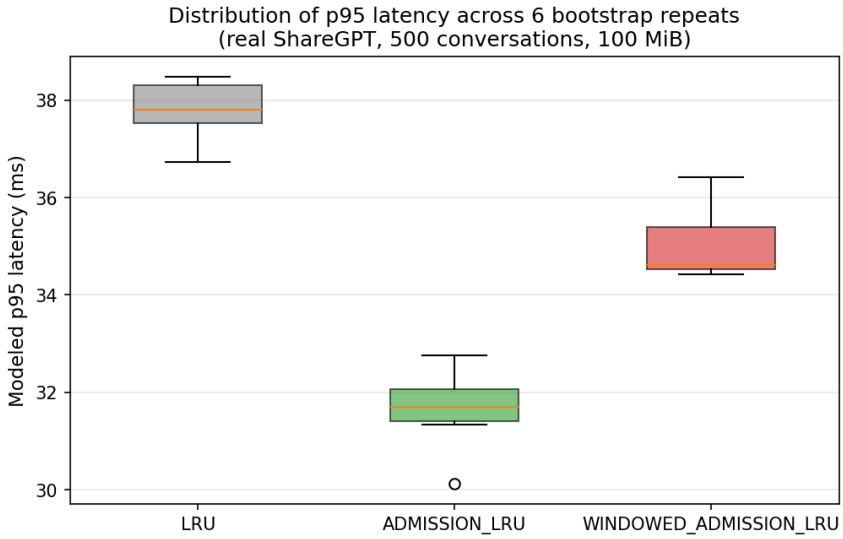

Figure 6. Distribution of p95 latency across the 6 bootstrap repeats (500 conversations, 100 MiB). Box shows quartiles; whiskers show the full range observed.

Beyond the mean, the spread across repeats matters for a deployment decision: a policy whose latency is consistently good is preferable to one with the same mean but occasional bad runs. Both admission-control variants are visibly tighter and lower than plain LRU here, not just lower on average -- consistent with fewer, more predictable evictions (Section 4.2) rather than an occasional lucky run driving the mean down.

5. Ablation Study

Each extension combines more than one idea. This section isolates them to attribute the results in Section 4 to specific mechanisms rather than to the design as an undifferentiated whole.

5.1 Direction A: is the cost-density term actually load-bearing?

CostAwareEvictionPolicy's score combines cost-density, recency decay, and frequency. Because the synthetic workloads use a uniform chunk size, cost-density degenerates to a constant multiple of recompute tokens under them, which cannot show whether the cost term does real discriminative work when memory size actually varies. A direct, isolated two-chunk check (benchmarks/cache_policy/robustness_sweep.py, check_size_heterogeneity) resolves this outside the simulator: with hit count held equal, two chunks of equal cost-density but different absolute memory size score identically (0.165346 both), confirming the term normalizes by size rather than penalizing large chunks outright; with size and hit count held equal but recompute cost raised 9x, the score raises by exactly 9.00x. The cost term is genuinely load-bearing, not overridden by the frequency term added later to fix the real-data weakness described in Section 2.2.

5.2 Direction B: halve_every sensitivity

The frequency sketch's one tunable, halve_every, controls how quickly popularity estimates decay. Figure 7 (left) sweeps it against both a dense-reuse workload (mixed_zipfian) and a long-reuse-horizon workload (multi_round_chat).

5.3 Direction C: window_capacity and promotion_threshold sensitivity

Figure 7 (right) sweeps both of Direction C's tunables. A correctness cross-check falls out of this sweep almost for free: at promotion_threshold=1, every window overflow is promoted (a key's sketch estimate is always at least 1 the moment it is inserted), so the pending-discard queue is always empty and eviction always defers to the inner policy -- the windowed design should, by construction, degenerate exactly to plain LRU at this one setting. The multi_round_chat bar for lenient_promotion (58.0%) matches plain LRU's 58.0% baseline to the digit, which is exactly the expected degenerate case, not a coincidence to be worried about -- in contrast to the same kind of exact match at the shipped default configuration, which was the symptom that led to discovering the zero-sum window bug described in Section 2.4.

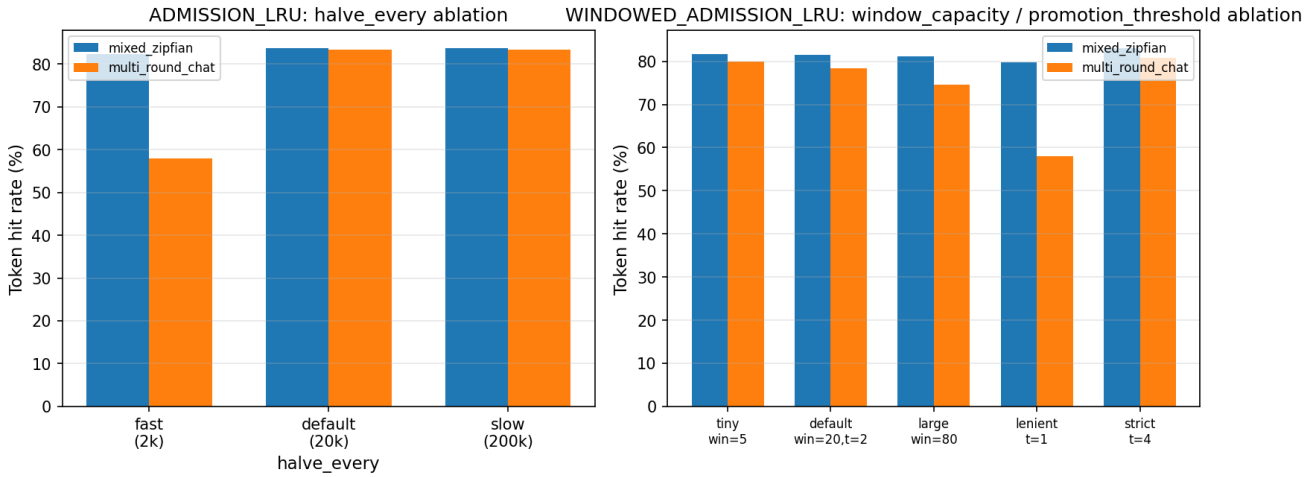

Figure 7. Left: ADMISSION_LRU's halve_every ablation. Right: WINDOWED_ADMISSION_LRU's window_capacity/promotion_threshold ablation.

Table 3. halve_every ablation, ADMISSION_LRU, 100 MiB.

Variant	mixed_zipfian	multi_round_chat
halve_every=2,000 (fast)	82.3%	58.0%
halve_every=20,000 (default)	83.7%	83.3%
halve_every=200,000 (slow)	83.7%	83.3%

On multi_round_chat, fast decay performs identically to no admission control at all (58.0%) -- the decay window is shorter than a conversation's round-to-round reuse gap, so a chunk's accumulated frequency credit is gone by the time it would be reused. Default and slow decay both fully recover the benefit (83.3%), and slow decay never underperforms default in this sweep -- the shipped 20,000 default is a reasonable but not universally optimal choice; a workload with an even longer reuse horizon could need it raised further.

Table 4. window_capacity/promotion_threshold ablation, WINDOWED_ADMISSION_LRU, 100 MiB.

Variant	mixed_zipfian	multi_round_chat
tiny window (5), t=2	81.6%	79.9%
default window (20), t=2	81.5%	78.3%
large window (80), t=2	81.1%	74.5%
lenient (20, t=1)	79.8%	58.0%
strict (20, t=4)	83.0%	80.8%
no admission control (LRU)	79.8%	58.0%

Smaller windows and stricter promotion both trend better -- less window capacity is spent holding entries

that never earn promotion, so strict_promotion (t=4) is the best-performing windowed variant on both workloads tested, though still short of ADMISSION_LRU's best numbers from Section 4.1.

5.4 Direction-level ablation: which idea contributed what

At the level of the whole design-space exploration, the three directions are themselves an ablation: Direction A isolates "add cost- and frequency-awareness to eviction ranking", Direction B isolates "add outright admission rejection on top of any ranking", and Direction C isolates "bound admission rejection's worst case with a windowed structure." Layering Direction B on top of Direction A (ADMISSION_COST_AWARE, composable today via get_cache_policy) rescues COST_AWARE's real-data weakness substantially (Table 2's COST_AWARE-family panel in Figure 4) but never catches up to ADMISSION_LRU -- admission control is the dominant idea, and cost-awareness is a genuine but strictly smaller contributor on the workloads tested.

6. Discussion

6.1 Trade-offs

No policy dominates on every axis measured. ADMISSION_LRU has the largest peak hit-rate and latency wins of anything tested, but Section 4.3 shows it can lose to doing nothing at all under generously-sized, low-pressure caches, and Figure 8 (below) shows it can freeze outright under one-shot-dominated traffic. WINDOWED_ADMISSION_LRU trades away a real fraction of that peak upside (Table 1, Table 2, Figure 5 all show it consistently between the baseline and the strict design) in exchange for a hard, structural guarantee that the freeze failure mode is unreachable. COST_AWARE's cost-density term is real (Section 5.1) but is dominated by frequency signal on real traffic, where almost every chunk is touched exactly once -- there is simply no reuse signal for a frequency-aware or cost-aware term to exploit on a majority of the corpus.

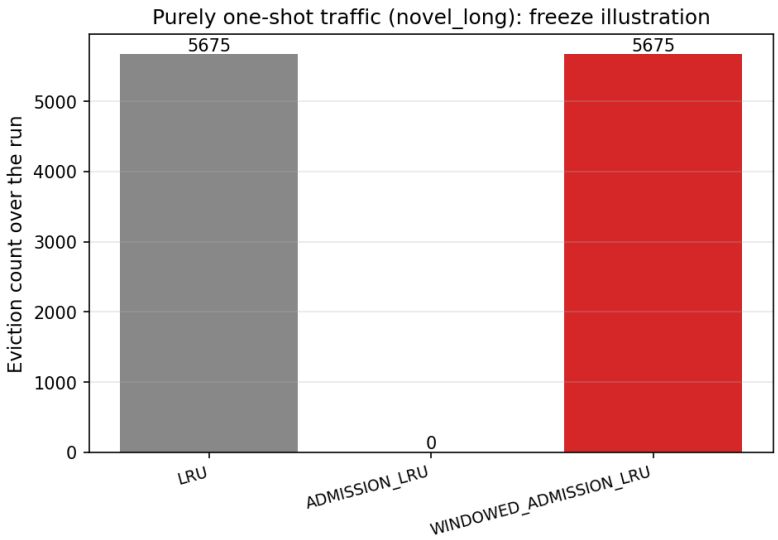

Figure 8. Purely one-shot traffic (novel_long, 500 documents, 2 MiB cache): ADMISSION_LRU's eviction count collapses to zero after the cache first fills (5,675 rejections instead), while plain LRU and WINDOWED_ADMISSION_LRU both keep evicting/rotating normally.

Figure 8's numbers are stark and worth stating precisely: under identical traffic, LRU and WINDOWED_ADMISSION_LRU both record 5,675 evictions, while ADMISSION_LRU records zero evictions and 5,675 rejections -- the cache fills once and then never changes for the rest of the run. This

does not affect hit rate for purely one-shot traffic (it is 0% for every policy there by construction), but it is a real, silent behavioral cliff for traffic that is mostly, but not entirely, one-shot: a realistic scenario in which the cache's useful capacity could shrink over time as more slots get permanently claimed by early, never-reused entries.

6.2 Sensitivity to parameters

The evaluation methodology throughout this report treats a single (policy, workload, cache-size) reading as insufficient evidence -- every result in Section 4 is a sweep, not a point estimate, precisely because parameter sensitivity turned out to matter in practice. halve_every must be matched to a workload's reuse horizon or admission control's entire benefit silently disappears (Table 3); window_capacity and promotion_threshold trade peak hit rate against the amount of window capacity spent on entries that never earn promotion (Table 4). None of these parameters have a value that is optimal across every workload tested, which argues for exposing them as tunable configuration rather than hardcoding the defaults used in this report.

6.3 On the evaluation methodology itself

Three real bugs were caught during this project, and none of them were caught by unit tests, type checking, or code review -- all three were caught by re-running the benchmark suite and distrusting an implausible number rather than accepting it. Bug 1 (Section 2.3) was caught because the shipped class scored worse than its own un-wrapped inner policy, the opposite of the validated prototype's result. Bug 2 (Section 2.3) was caught by a crash the first time a specific inner policy (LFU) was wrapped -- a combination the original thin verification never exercised. Bug 3 (Section 2.4) is the most instructive: it caused no crash and no test failure, only a policy that was silently a no-op, caught solely because two eviction counts that should have been different were identical to the exact digit. This is the practical argument for the full sweep-and-ablation methodology this report follows, rather than a single before/after benchmark reading: a design can look correct, pass every test written for it, and still be doing nothing.

7. Conclusion and Future Work

Admission control -- rejecting a low-value newcomer outright instead of only re-ranking eviction order -- is the strongest and most general improvement evaluated in this project, and it is not specific to any one inner ranking policy. Cost-awareness is a real, measurable improvement to eviction ranking specifically, but on real conversational traffic its available signal is dominated by the same frequency information admission control already exploits more directly. Neither admission-control variant is a strict improvement over the other: the strict design has the larger peak upside and a documented catastrophic failure mode under one-shot-dominated traffic; the windowed design removes that failure mode structurally at the cost of a real fraction of the peak upside, and converges toward the baseline rather than toward the strict design's wins under traffic close enough to purely one-shot.

The most unexpected finding was less about any one policy and more about the real data itself: real multi-turn conversational traffic (ShareGPT) is far more one-shot-per-chunk than any of the synthetic workloads designed to approximate it, including the Zipf-skewed one -- which is exactly why Direction C's windowed design was needed at all, and why it converges toward plain LRU rather than toward Direction B's wins at the largest, most realistic corpus scales tested. A synthetic benchmark suite, however carefully designed, is not a substitute for validating against real traffic before drawing a general

conclusion.

Future work, kept grounded to what the current results actually support: (1) wire should_admit into a real storage backend -- local_disk_backend.py's submit_put_task was identified as the lowest-risk integration point, since it already has both the key and the required size available before its eviction loop runs, unlike local_cpu_backend.py's allocate(), which never sees the key at all; (2) since no single policy dominates, expose the policy choice as deployment-level configuration rather than picking one default, informed by an estimate of how one-shot-dominated the target traffic is; (3) investigate whether window_capacity and promotion_threshold could be tuned adaptively from an online estimate of the workload's reuse rate, to recover more of Direction B's peak upside without reintroducing its freeze risk -- speculative, and not attempted in this report.

8. Appendix: Code and Data Artifacts

A.1 Policy implementations

```
lmcache/v1/storage_backend/cache_policy/base_policy.py         BaseCachePolicy interface (incl.
should_admit hook)
lmcache/v1/storage_backend/cache_policy/lru.py, lfu.py,
  fifo.py, mru.py                                             Baseline policies
lmcache/v1/storage_backend/cache_policy/cost_aware_policy.py  Direction A
lmcache/v1/storage_backend/cache_policy/admission_control.py        Directions    B    and    C
(AdmissionControlledPolicy,

WindowedAdmissionControlledPolicy, shared sketch)
lmcache/v1/storage_backend/cache_policy/__init__.py            get_cache_policy factory, prefix
composition
```

A.2 Benchmark tooling

```
lmcache/tools/cache_policy_bench/runner.py       Simulator, sweep driver, CSV/JSON writers
lmcache/tools/cache_policy_bench/workloads.py    Synthetic request generators
lmcache/tools/cache_policy_bench/cost_model.py   Modeled latency function
lmcache/tools/cache_policy_bench/sharegpt_workload.py
                                                  Real ShareGPT corpus adapter
benchmarks/cache_policy/run_ablation.py           Direction A ablation
benchmarks/cache_policy/run_admission_control_ablation.py
                                                    Directions B/C ablation (this report's Table
3/4)
benchmarks/cache_policy/robustness_sweep.py       Zipf sweep + cost-density isolation check
benchmarks/cache_policy/real_dataset_eval.py      Bootstrap-CI real-data validation
benchmarks/cache_policy/stats.py                  Percentile-bootstrap CI helper
benchmarks/cache_policy/report/generate_figures.py
                                                  Regenerates every figure in this report
benchmarks/cache_policy/report/build_report.py    Regenerates this PDF
```

A.3 Tests

```
tests/v1/test_cache_policy.py                     Correctness tests, all policies
tests/benchmarks/test_cache_policy_bench.py        Synthetic smoke + regression-locking tests
tests/benchmarks/test_cache_policy_bench_real_data.py
                                                  Opt-in real-data stress tests
```

A.4 Data artifacts

```
benchmarks/cache_policy/results/admission_control/sweep_results.json
benchmarks/cache_policy/results/admission_control/
  admission_control_ablation.json,
  windowed_admission_control_ablation.json
benchmarks/cache_policy/results/admission_control/robustness_zipf_skew.json
benchmarks/cache_policy/results/real_data/
  real_dataset_ci.json, real_dataset_raw.json
```

A.5 Design documents

```
docs/design/v1/storage_backend/cache_policy/cost-aware-policy-eval.md
docs/design/v1/storage_backend/cache_policy/admission-control-policy.md
```

A.6 Reproducing this report

See benchmarks/cache_policy/README.md for environment setup and instructions to re-run the full benchmark suite. Given a prepared environment and the ShareGPT corpus at benchmarks/multi_round_qa/ShareGPT.json, this report's figures and PDF are regenerated with:

```
python benchmarks/cache_policy/report/generate_figures.py
python benchmarks/cache_policy/report/build_report.py
```